Taller 11: Domain-Driven Design (DDD)



Universidad del Cauca

Vigilada Mineducación

Laboratorio de Ingeniería de Software II

Modelado estratégico y táctico en el Banco de Preguntas Saber Pro: contextos delimitados, lenguaje ubicuo, agregados, entidades, objetos de valor e invariantes en Java puro

Presentado por:

Yeferson Santiago Caicedo Orozco

Iván Alexander López Lasso

Adrián Camilo Araujo Urbano

Profesora:

Ing. Paola Andrea Bedoya Toro

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Programa de Ingeniería de Sistemas

Popayán, noviembre de 2026

CONTENIDO

Introducción **1**

Capítulo 1: Modelado estratégico **2**

1. Contextos delimitados y lenguaje ubicuo **2**

2. Mapa de contextos **2**

3. Respuestas del modelado estratégico **3**

3.1. ¿Por qué separar el sistema en estos cuatro contextos? 3
3.2. ¿Qué significa «Pregunta» en cada contexto? 4
3.3. ¿Cuál es el dominio central (core domain)? 4
3.4. ¿Cómo se protege la frontera entre contextos? 4

Capítulo 2: Modelado táctico **5**

4. Estructura de paquetes del dominio **5**

5. Bloques de construcción **5**

5.1. Objetos de valor frente a entidades 6
5.2. Invariantes de negocio del agregado 6

6. Diagrama de clases del dominio **6**

7. Ciclo de vida de la pregunta **7**

8. Fragmentos de código **8**

8.1. Objeto de valor Justificacion 8
8.2. Raíz de agregado Pregunta: envío a revisión 9
8.3. Entidad Revision: dictamen 9

Capítulo 3: Pruebas y aislamiento del dominio **10**

9. Pruebas unitarias con JUnit 5 ..... **10**

10. Aislamiento del dominio **11**

Capítulo 4: Entregables **12**

11. Repositorio y ejecución **12**

12. Conclusiones **12**

13. Referencias **12**

LISTA DE ILUSTRACIONES Y TABLAS

Ilustración 1 . Mapa de contextos: relaciones entre Autoría, Validación, Ciclo de Vida y Revisión por Pares. 3
Ilustración 2 . Diagrama de clases del dominio: agregado Pregunta, objetos de valor, entidades de revisión, repositorio y excepciones. 7

Ilustración 3 . Diagrama de estados de la pregunta con las guardas que impone el agregado. 8

Tabla 1 . Contextos delimitados del Banco de Preguntas Saber Pro y su ubicación en el proyecto. 2
Tabla 2 . Clasificación de las clases del dominio según los bloques de construcción de DDD. 5
Tabla 3 . Pruebas unitarias del módulo de dominio. 10

Introducción

Este informe presenta la solución del **Taller 11: Domain-Driven Design (DDD)** sobre el sistema del Banco de Preguntas Saber Pro. El trabajo tiene dos partes: el **modelado estratégico**, que divide el problema en contextos delimitados con su propio lenguaje ubicuo, y el **modelado táctico**, que lleva las reglas del negocio a objetos de dominio con comportamiento: una raíz de agregado, entidades, objetos de valor, excepciones de negocio y un contrato de repositorio.

El código está en el módulo Maven `saberpro-domain` del repositorio del proyecto. Es Java puro (Java 17 o superior): el módulo no hereda de Spring Boot ni declara JPA, y su única dependencia es JUnit 5 para las pruebas. Una prueba automática recorre las fuentes del dominio y falla si alguna clase importa un framework técnico.

El trabajo lo realizamos tres integrantes, con autorización de la profesora: Yeferson Santiago Caicedo Orozco, Iván Alexander López Lasso y Adrián Camilo Araujo Urbano.

Capítulo 1: Modelado estratégico

1. Contextos delimitados y lenguaje ubicuo

Partimos de los cuatro contextos que propone el taller y los ubicamos en el proyecto. Cada contexto tiene una responsabilidad, un vocabulario propio y, en la arquitectura del corte 2, un microservicio que lo implementa. En el módulo de dominio del taller, los contextos de Autoría, Ciclo de Vida y Revisión por Pares quedan representados por clases concretas; la Validación aparece como las invariantes que el agregado comprueba antes de enviar una pregunta a revisión.

Tabla 1. Contextos delimitados del Banco de Preguntas Saber Pro y su ubicación en el proyecto.

Contexto	**Responsabilidad principal**	**Lenguaje ubicuo**	**Dónde vive en el proyecto**
Autoría	Gestión de reactivos DCE: enunciados, distractores, clave y justificación.	Pregunta, Enunciado, Clave, Justificacion, Autor, OpcionRespuesta	`question-service`; en el taller: `Pregunta`, `Enunciado`, `Justificacion`, `OpcionRespuesta`
Validación	Verificación de reglas de calidad estructural y pedagógica.	ReglaEstructural, FiltroExpresion, ValidezOpciones	`validation-service`; en el taller: invariantes de envío y palabras prohibidas de la justificación
Ciclo de Vida	Control estricto de los estados y transiciones de las preguntas.	EstadoPregunta, Transicion, Borrador, PendienteRevision, EnRevision, Aprobada, Rechazada	`lifecycle-service`; en el taller: `EstadoPregunta` y las transiciones de `Pregunta`
Revisión por Pares	Asignación de evaluadores, dictamen y observaciones pedagógicas.	Revisor, Asignacion, Evaluacion, Dictamen, Observacion	`review-service`; en el taller: `Revisor`, `Revision`, `Dictamen`

2. Mapa de contextos

La Ilustración 1 muestra cómo se relacionan los contextos. Las flechas indican quién depende de quién y por qué medio se comunican.

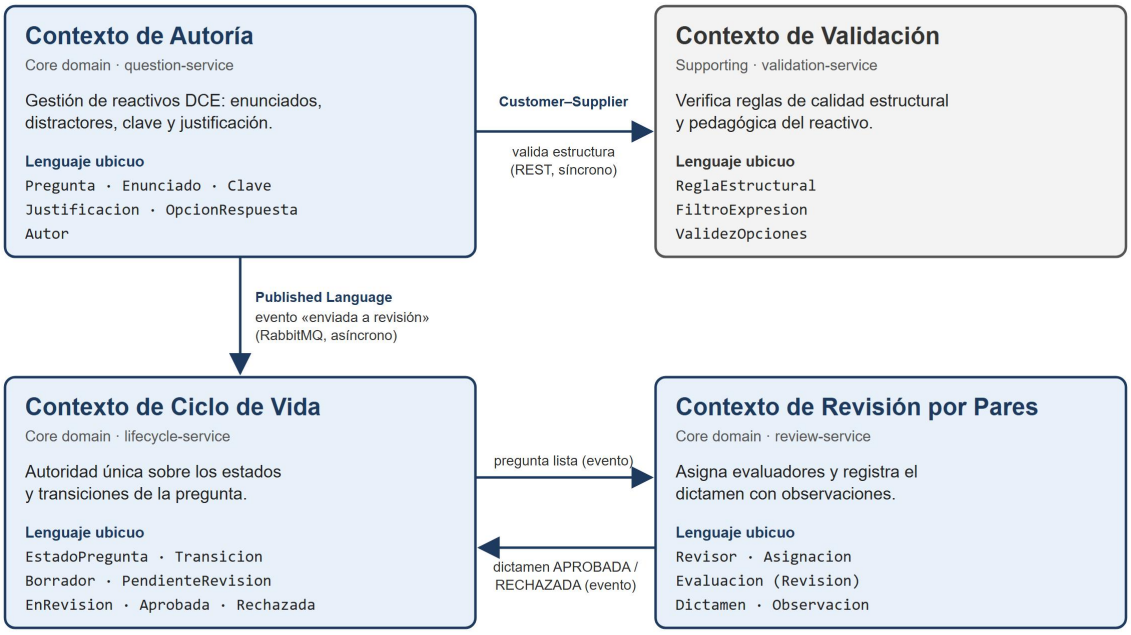


Ilustración 1. Mapa de contextos: relaciones entre Autoría, Validación, Ciclo de Vida y Revisión por Pares.

- **Autoría → Validación (Customer–Supplier):** Autoría es el cliente; antes de aceptar cambios consulta a Validación de forma síncrona por REST, porque necesita la respuesta en el momento.
- **Autoría → Ciclo de Vida (Published Language):** el autor no cambia el estado por su cuenta. Publica el evento de envío a revisión por RabbitMQ y Ciclo de Vida, como autoridad única, aplica la transición.
- **Ciclo de Vida ↔ Revisión por Pares:** Ciclo de Vida avisa que una pregunta está lista; Revisión asigna evaluadores y devuelve el dictamen (APROBADA o RECHAZADA), que vuelve a Ciclo de Vida para fijar el estado final.

3. Respuestas del modelado estratégico

3.1. ¿Por qué separar el sistema en estos cuatro contextos?

Porque cada uno cambia por razones distintas y lo usan personas distintas. El autor redacta y corrige reactivos; las reglas de calidad cambian cuando el comité ajusta los criterios DCE; el flujo de estados es una política institucional que no debe depender del autor; y la revisión por pares tiene sus propias reglas (quién puede revisar, cómo se emite un dictamen). Separarlos evita que un cambio en una de estas áreas obligue a tocar las otras.

3.2. ¿Qué significa «Pregunta» en cada contexto?

La misma palabra tiene un modelo distinto según el contexto, y eso es justamente lo que delimita cada uno:

- **Autoría:** un reactivo editable con enunciado, cuatro opciones, una clave y una justificación.
- **Validación:** un conjunto de textos que se somete a reglas estructurales y de expresión.
- **Ciclo de Vida:** un identificador con un estado y las transiciones permitidas desde él.
- **Revisión por Pares:** el objeto de una evaluación; a `Revision` le basta el ID de la pregunta y el ID de su autor.

3.3. ¿Cuál es el dominio central (core domain)?

Autoría, Ciclo de Vida y Revisión por Pares forman el núcleo: son lo que diferencia al banco de preguntas de cualquier otro sistema. Validación es un dominio de soporte; es necesaria, pero sus reglas podrían reutilizarse o reemplazarse sin cambiar la esencia del negocio.

3.4. ¿Cómo se protege la frontera entre contextos?

Los contextos no comparten objetos: se comunican por contratos (REST) y por eventos (RabbitMQ). Dentro del dominio esto se ve en que `Revision` guarda solo `preguntaId` y no una referencia al objeto `Pregunta`; así, cambiar el agregado de Autoría no rompe el modelo de Revisión.

Capítulo 2: Modelado táctico

4. Estructura de paquetes del dominio

El módulo sigue la estructura de paquetes que pide el taller, con dos entidades adicionales (`Revisor` y `Revision`) y un enumerado de dictamen para el contexto de Revisión por Pares:

```
saberpro-domain/           (pom.xml: solo JUnit 5, alcance test)
└── co.edu.unicauca.saberpro.domain/
    ├── model/
    │   ├── Pregunta.java          AGGREGATE ROOT
    │   ├── Enunciado.java         VALUE OBJECT (inmutable)
    │   ├── Justificacion.java     VALUE OBJECT (inmutable)
    │   ├── OpcionRespuesta.java   VALUE OBJECT (inmutable)
    │   ├── Revisor.java           ENTITY (ID de docente)
    │   ├── Revision.java          ENTITY (ID de evaluación)
    │   ├── EstadoPregunta.java    ENUM (BORRADOR, PENDIENTE_REVISION...)
    │   └── Dictamen.java          ENUM (APROBADA, RECHAZADA)
    ├── exception/
    │   ├── DomainException.java
    │   └── ReglaNegocioException.java
    └── repository/
        ├── PreguntaRepository.java        interfaz pura
        └── PreguntaMemoryRepository.java  implementación en memoria
```

5. Bloques de construcción

Tabla 2. Clasificación de las clases del dominio según los bloques de construcción de DDD.

Clase	Bloque DDD	Qué garantiza
`Pregunta`	Raíz de agregado	Única puerta para modificar el agregado. Protege las invariantes de envío y las transiciones de estado. Expone las opciones como lista inmodificable.
`Enunciado`	Objeto de valor	No nulo y con al menos 10 caracteres significativos. Inmutable; igualdad por valor.
`Justificacion`	Objeto de valor	Al menos 15 caracteres y sin palabras prohibidas («obvio», «obviamente», «evidentemente», «claramente», «porque sí», «todas las anteriores», «ninguna de las anteriores»), sin importar mayúsculas ni tildes.
`OpcionRespuesta`	Objeto de valor	Texto no vacío e indicador de clave (`esCorrecta`). Inmutable.
`Revisor`	Entidad	Identidad por ID de docente: dos revisores con el mismo ID son el mismo aunque cambie el nombre.
`Revision`	Entidad	Identidad por ID de evaluación. El autor no puede revisar su propia pregunta; el dictamen se emite una sola vez y un rechazo exige observación.
`EstadoPregunta`, `Dictamen`	Enumerados	Vocabulario cerrado del ciclo de vida y del veredicto.

Clase	Bloque DDD	Qué garantiza
`DomainException`, `ReglaNegocioException`	Excepciones de negocio	Toda regla violada se reporta con un mensaje del negocio, no con un error técnico.
`PreguntaRepository`	Repositorio (interfaz)	Contrato de persistencia del agregado expresado en el lenguaje del dominio, sin JPA.

5.1. Objetos de valor frente a entidades

La diferencia está en cómo se comparan. `Justificacion` es un objeto de valor: dos justificaciones con el mismo texto son intercambiables, por eso `equals` compara el texto y la clase es `final` con campos `final`. `Revisor` es una entidad: lo que importa es quién es, no cómo se llama, por eso `equals` solo usa el ID de docente y el nombre puede cambiar con `cambiarNombre`.

5.2. Invariantes de negocio del agregado

- Solo se agregan o modifican opciones, enunciado y justificación en estado `BORRADOR`.
- Una pregunta no puede tener más de 4 opciones; la quinta lanza `ReglaNegocioException`.
- Para enviarla a revisión debe tener enunciado y justificación válidos, **exactamente 4 opciones** y **exactamente 1 clave**.
- Las transiciones siguen un orden fijo: `BORRADOR` → `PENDIENTE_REVISION` → `EN_REVISION` → `APROBADA` | `RECHAZADA`; rechazar exige un motivo.

6. Diagrama de clases del dominio

La Ilustración 2 muestra el agregado `Pregunta` con su frontera de consistencia, las entidades de revisión, el repositorio y las excepciones.

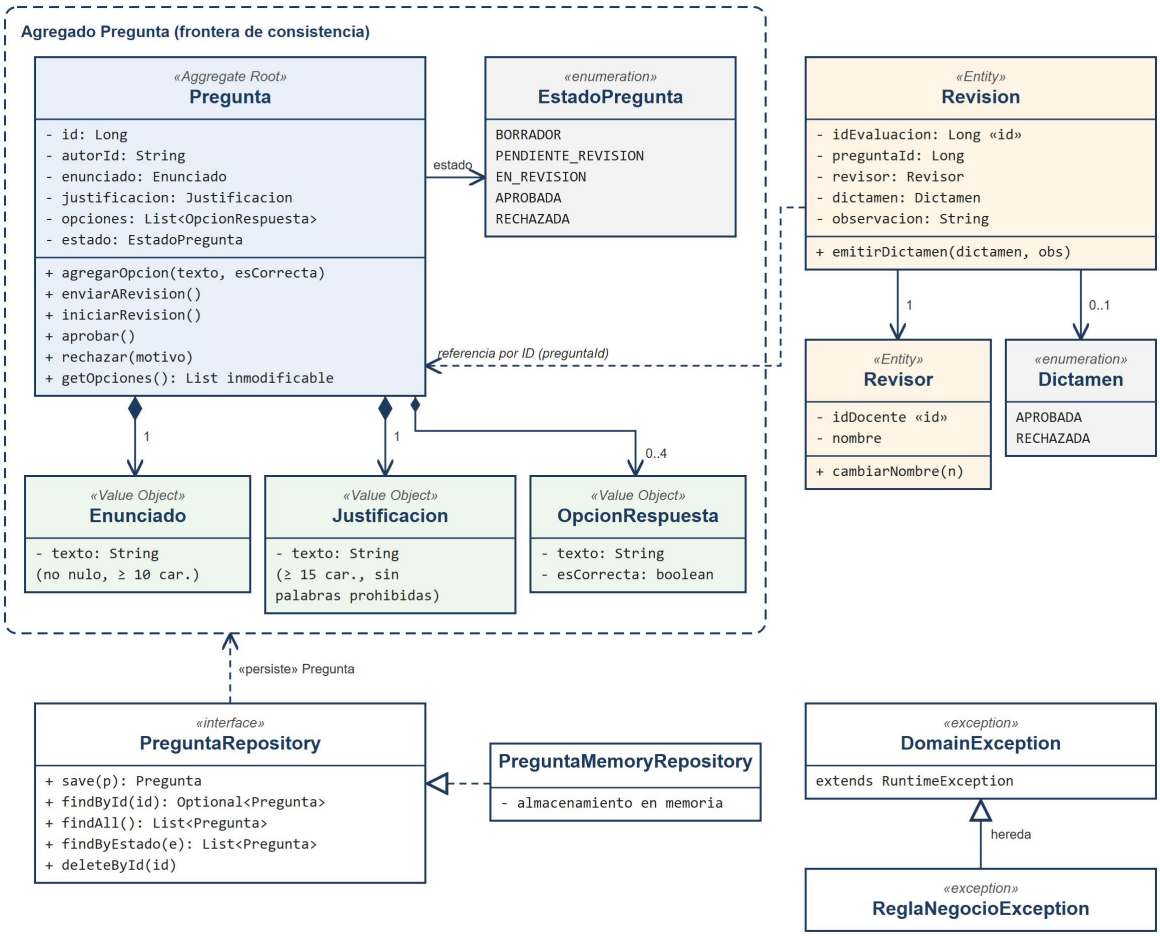


Ilustración 2. Diagrama de clases del dominio: agregado Pregunta, objetos de valor, entidades de revisión, repositorio y excepciones.

7. Ciclo de vida de la pregunta

La Ilustración 3 muestra los estados de la pregunta y las condiciones que el agregado verifica en cada transición.

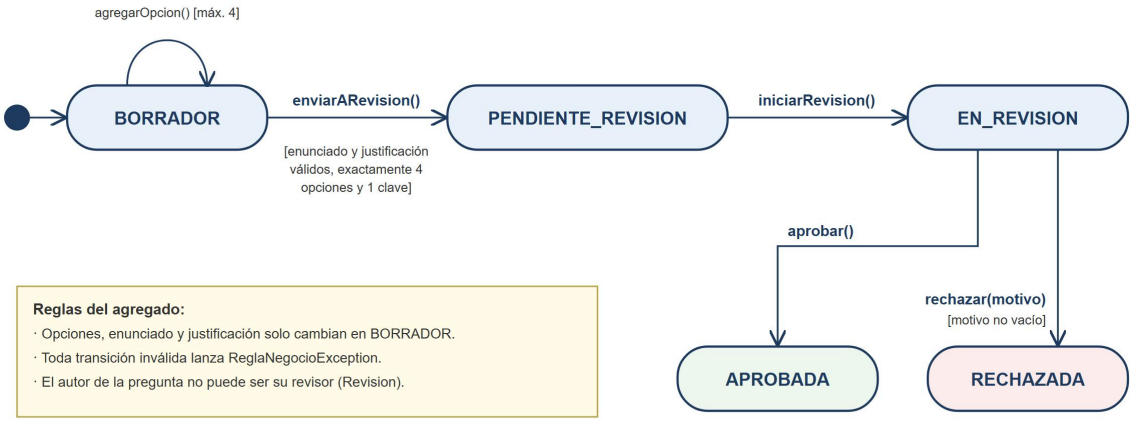


Ilustración 3. Diagrama de estados de la pregunta con las guardas que impone el agregado.

8. Fragmentos de código

8.1. Objeto de valor Justificacion

```
public final class Justificacion {
    public static final int LONGITUD_MINIMA = 15;
    public static final List<String> PALABRAS_PROHIBIDAS = List.of(
            "obvio", "obviamente", "evidentemente", "claramente",
            "porque si", "todas las anteriores", "ninguna de las anteriores");
    private final String texto;

    public Justificacion(String texto) {
        if (texto == null || texto.trim().length() < LONGITUD_MINIMA) {
            throw new ReglaNegocioException(
                "La justificación debe contener al menos 15 caracteres.");
        }
        String normalizado = normalizar(texto);   // minúsculas y sin tildes
        for (String prohibida : PALABRAS_PROHIBIDAS) {
            if (Pattern.compile("\\b" + Pattern.quote(prohibida) + "\\b")
                       .matcher(normalizado).find()) {
                throw new ReglaNegocioException(
                    "La justificación contiene la expresión prohibida \"" + prohibida + "\".");
            }
        }
        this.texto = texto.trim();
    }
}
```

8.2. Raíz de agregado Pregunta: envío a revisión

```
public void agregarOpcion(String texto, boolean esCorrecta) {
    if (this.estado != EstadoPregunta.BORRADOR) {
        throw new ReglaNegocioException("Solo se pueden modificar opciones en estado Borrador.");
    }
    if (this.opciones.size() >= 4) {
        throw new ReglaNegocioException("Una pregunta no puede tener más de 4 opciones.");
    }
    this.opciones.add(new OpcionRespuesta(texto, esCorrecta));
}

public void enviarARevision() {
    if (this.estado != EstadoPregunta.BORRADOR) {
        throw new ReglaNegocioException("Solo una pregunta en estado Borrador puede ser enviada a
revisión.");
    }
    validarInvariantesParaEnvio();   // enunciado, justificación, 4 opciones, 1 clave
    this.estado = EstadoPregunta.PENDIENTE_REVISION;
}
```

8.3. Entidad Revision: dictamen

```
public Revision(Long idEvaluacion, Pregunta pregunta, Revisor revisor) {
    ...
    if (revisor.getIdDocente().equals(pregunta.getAutorId())) {
        throw new ReglaNegocioException("El autor de una pregunta no puede ser su revisor.");
    }
    this.preguntaId = pregunta.getId();   // referencia por ID, no por objeto
}

public void emitirDictamen(Dictamen dictamen, String observacion) {
    if (this.dictamen != null) throw new ReglaNegocioException("La revisión ya tiene un dictamen
emitido.");
    if (dictamen == Dictamen.RECHAZADA && (observacion == null || observacion.isBlank())) {
        throw new ReglaNegocioException("Un rechazo debe incluir una observación pedagógica.");
    }
    ...
}
```

Capítulo 3: Pruebas y aislamiento del dominio

9. Pruebas unitarias con JUnit 5

Las pruebas no usan Spring, base de datos ni simuladores: crean objetos del dominio y comprueban que las reglas se cumplen o que se lanza `ReglaNegocioException`.

Tabla 3. Pruebas unitarias del módulo de dominio.

Clase de prueba	Pruebas	Qué verifica
`PreguntaAggregateTest`	9	Envío sin 4 opciones, quinta opción, ninguna clave o varias claves, envío sin enunciado o sin justificación, cambio de opciones después del envío, envío exitoso y los dos ciclos de vida completos (aprobación y rechazo con motivo).
`ValueObjectsTest`	6	Longitud mínima de `Justificacion` y `Enunciado`, palabras prohibidas, igualdad por valor y validación de `OpcionRespuesta`.
`EntidadesRevisionTest`	3	Igualdad por identidad de `Revisor`; el autor no puede revisar; dictamen único y rechazo con observación.
`PreguntaMemoryRepositoryTest`	3	Guardar con ID automático y recuperar, filtrar por estado y eliminar en el repositorio en memoria.
`DominioPuroTest`	1	Ninguna clase del dominio importa Spring, JPA, Hibernate ni Lombok.
Total	**22**	**Todas en verde.**

Prueba pedida en el taller (paso 3):

```
@Test
void noDebePermitirEnviarARevisionSinCuatroOpciones() {
    Pregunta pregunta = new Pregunta();
    pregunta.agregarOpcion("Opción A", true);
    assertThrows(ReglaNegocioException.class, pregunta::enviarARevision);
}
```

Resultado de `mvn test` en `saberpro-domain`:

```
[INFO] Running co.edu.unicauca.saberpro.domain.DominioPuroTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running co.edu.unicauca.saberpro.domain.model.EntidadesRevisionTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running co.edu.unicauca.saberpro.domain.model.PreguntaAggregateTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running co.edu.unicauca.saberpro.domain.model.ValueObjectsTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running co.edu.unicauca.saberpro.domain.repository.PreguntaMemoryRepositoryTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

10. Aislamiento del dominio

El aislamiento se garantiza en tres niveles:

- **Compilación:** el `pom.xml` de `saberpro-domain` no tiene padre de Spring Boot y solo declara `junit-jupiter` con alcance `test`. Si una clase del dominio importara Spring o JPA, el módulo no compilaría.
- **Prueba:** `DominioPuroTest` recorre las fuentes del dominio y falla si encuentra imports de `org.springframework`, `jakarta.persistence`, `javax.persistence`, `org.hibernate` o `lombok`.
- **Integración continua:** GitHub Actions tiene un job propio, «Dominio DDD puro (taller 11)», que compila el módulo y corre sus pruebas en cada push y pull request a `main`.

```
<dependencies>
    <!-- JUnit 5: única dependencia, y solo en pruebas -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Capítulo 4: Entregables

11. Repositorio y ejecución

Repositorio: https://github.com/santyxswc/Banco_preguntas-ISoftII

Módulo del taller: saberpro-domain/

Para ejecutar las pruebas se necesita Java 17 o superior y Maven:

```
cd saberpro-domain
mvn test
```

La documentación del taller en el repositorio está en `TALLER11_DDD_DOCUMENTACION.md` y en `saberpro-domain/README.md`.

12. Conclusiones

- Delimitar contextos aclaró que «Pregunta» no es un único modelo: cada contexto necesita una vista distinta, y forzar un modelo común acoplaría partes que cambian por razones distintas.
- Poner las reglas en el agregado `Pregunta` y en los objetos de valor hace imposible construir un estado inválido: no hace falta validar en cada servicio, porque el objeto se protege a sí mismo.
- Separar el dominio en un módulo sin frameworks permitió probar todas las reglas del negocio con JUnit puro, en milisegundos y sin base de datos.
- La diferencia entre entidad y objeto de valor quedó clara en el código: `Revisor` se compara por su ID; `Justificacion`, por su contenido.

13. Referencias

- Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- Vernon, V. (2013). Implementing Domain-Driven Design. Addison-Wesley.
- Bedoya Toro, P. A. (2026). Taller 11: Domain-Driven Design (DDD). Laboratorio de Ingeniería de Software II, Universidad del Cauca.